

Sprawozdanie z realizacji projektu I (OpenMP, Vtune)

Laboratorium z przetwarzania równoległego

Autorzy: Kamil Dobek 160134, Michał Leszczyński 160074
Grupa dziekańska: L9
Termin zajęć: Wtorek, 9:45
Termin oddania: 05.05.2026
Wersja sprawozdania: 1.0
Kontakt e-mail: kamil.dobek@student.put.poznan.pl, michal.leszczynski@student.put.poznan.pl

Opis zadania

Realizowane zadanie polega na implementacji i analizie wydajnościowej algorytmów wyszukujących liczby pierwsze w zadanym przedziale. Analiza obejmuje wersje sekwencyjne oraz zrównoleglone z wykorzystaniem dyrektyw OpenMP, ze szczególnym uwzględnieniem lokalności dostępu do danych oraz podziału pracy na rdzenie procesora hybrydowego.

Punkt 1: Opis systemu obliczeniowego

Wykorzystany system obliczeniowy

Badania przeprowadzono dwuetapowo na następującej platformie sprzętowo-programowej:

- **Procesor:** Intel Core i5-12500H (architektura hybrydowa Alder Lake).
- **Rdzenie:** 12 rdzeni fizycznych (4 Performance-cores, 8 Efficient-cores), 16 procesorów logicznych.
- **Pamięć podręczna:** Cache L1: 80 KB (na rdzeń), Cache L2: 1.25 MB (na rdzeń), E-Core L1: 96 KB (na rdzeń), E-Core L2: 2 MB (na klaster), Cache L3: 18 MB (współdzielona).
- **Bufory translacji adresów:** Procesor wykorzystuje wielopoziomowy mechanizm translacji adresów obejmujący bufory TLB pierwszego poziomu oraz dalszy poziom translacji w postaci STLB. W architekturze hybrydowej rdzenie P-core i E-core mają odrębne struktury wykonawcze, dlatego koszty translacji adresów mogą różnić się w zależności od typu rdzenia i charakteru dostępu do danych.
- **Środowisko natywne (Pomiary sprzętowe):** System Windows 11, kompilator Intel C++ Compiler (icx), Intel(R) VTune(TM) Profiler 2025.10.0.

Punkt 2: Opis wariantów kodu

W ramach projektu zaimplementowano siedem wariantów podstawowych algorytmów (k1, k2, k3, k3a, k4, k4a, k5), różniących się podejściem do algorytmu, sposobem podziału pracy oraz lokalnością dostępu do danych. Dodatkowo przygotowano trzy zmodyfikowane wersje: k2_chunk, k5b oraz k5c.

Wariant k1: Sekwencyjne dzielenie

Wariant bazowy, w którym każda liczba z badanego zakresu jest sprawdzana sekwencyjnie przez dzielenie przez liczby pierwsze nie większe niż pierwiastek z badanej liczby.

```
1 long long m = 50000000, n = 100000000;  
2 long long range = n - m + 1;  
3  
4 bool* result = (bool*)malloc(range * sizeof(bool));  
5 memset(result, true, range * sizeof(bool));  
6  
7 long long limit = (long long)sqrt(n);  
8 bool* primeArray = (bool*)malloc((limit + 1) * sizeof(bool));  
9 memset(primeArray, true, (limit + 1) * sizeof(bool));  
10  
11 for (long long i = 2; i * i <= n; i++) {  
12     for (long long j = 2; j * j <= i; j++) {  
13         if (primeArray[j] && i % j == 0) {  
14             primeArray[i] = false;  
15             break;  
16         }  
17     }  
18 }
```

```

17
18 for (long long i = m; i <= n; i++) {
19     for (long long j = 2; j * j <= i; j++) {
20         if (primeArray[j] && i % j == 0) {
21             result[i - m] = false;
22             break;
23         }
24     }
25 }

```

Listing 1: Inicjalizacja tablic i sekwencyjne dzielenie (k1)

- **Znaczenie zmiennych:** Zmienne `m` i `n` określają badany zakres, a `range = n - m + 1` liczbę sprawdzanych wartości. Tablica `result` przechowuje wynik dla każdej liczby z zakresu, przy czym `result[i-m]` odpowiada liczbie `i`. Tablica `primeArray` zawiera pomocniczą informację o liczbach pierwszych do wartości `sqrt(n)`.
- **Przebieg przetwarzania:** Najpierw tworzona jest tablica `primeArray`, zawierająca potencjalne dzielniki pierwsze. Następnie każda liczba `i` z zakresu `<m,n>` jest dzielona przez kolejne wartości `j`. Jeżeli znaleziony zostanie dzielnik, liczba zostaje oznaczona jako złożona przez ustawienie `result[i-m] = false`, a dalsze sprawdzanie tej liczby jest przerywane instrukcją `break`.
- **Podział pracy i przydział zadań:** Wariant nie dzieli pracy pomiędzy wątki. Całość przetwarzania wykonuje jeden wątek. Pojedynczym zadaniem logicznym jest sprawdzenie jednej liczby z zakresu.
- **Szeregowanie i zrównoważenie:** Nie występuje szeregowanie OpenMP. Koszt sprawdzenia pojedynczej liczby jest zmienny: liczby złożone z małym dzielnikiem kończą sprawdzanie szybko, natomiast liczby pierwsze wymagają przejścia przez większą część pętli wewnętrznej.
- **Dyrektywy OpenMP:** Wariant nie wykorzystuje dyrektyw zrównoleglających OpenMP.
- **Problemy poprawnościowe:** Dla badanych zakresów kod zwraca poprawne wyniki. Sprawdzanie dzielników do pierwiastka z badanej liczby jest wystarczające do określenia, czy liczba jest pierwsza.
- **Problemy efektywnościowe:** Głównym ograniczeniem jest duża liczba operacji modulo w pętli wewnętrznej. Czas wykonania zależy od liczby badanych wartości oraz od tego, jak szybko dla liczb złożonych zostanie znaleziony dzielnik.

Wariant k2: Równoległe dzielenie z dynamicznym przydziałem iteracji

Zmodyfikowana wersja wariantu k1, w której zrównoleglono pętlę po badanych liczbach przy użyciu dyrektywy `#pragma omp parallel for schedule(dynamic)`.

```

1 #pragma omp parallel for schedule(dynamic)
2 for (long long i = m; i <= n; i++) {
3     for (long long j = 2; j * j <= i; j++) {
4         if (primeArray[j] && i % j == 0) {
5             result[i - m] = false;
6             break;
7         }
8     }
9 }

```

Listing 2: Równoległa pętla dzielenia (k2)

- **Znaczenie zmiennych:** Zmienne i tablice mają takie samo znaczenie jak w wariantcie k1. Tablica `primeArray` jest przygotowywana przed częścią równoległą, a następnie tylko odczytywana przez wątki. Tablica `result` jest współdzielona, ale każda iteracja zapisuje inny element `result[i-m]`.
- **Podział pracy i przydział zadań:** Pojedynczym zadaniem jest sprawdzenie jednej liczby `i`. Liczba zadań jest równa liczbie elementów zakresu. Zadania są przydzielane wątkom dynamicznie podczas wykonywania pętli.
- **Szeregowanie i zrównoważenie:** Zastosowano `schedule(dynamic)`, ponieważ koszt sprawdzenia kolejnych liczb nie jest jednakowy. Dynamiczny przydział iteracji ogranicza ryzyko nierównomiernego obciążenia wątków, ale wprowadza dodatkowy narzut związany z przydzielaniem pracy przez środowisko OpenMP.
- **Dyrektywy OpenMP:** Dyrektywa `#pragma omp parallel for schedule(dynamic)` tworzy zespół wątków i rozdziela iteracje pętli pomiędzy nie. Indeks pętli `i` jest prywatny dla iteracji, a tablice `result` i `primeArray` są współdzielone. Na końcu pętli występuje niejawna bariera synchronizacyjna.
- **Problemy poprawnościowe:** Wariant jest poprawny współbieżnie, ponieważ każda liczba `i` jest sprawdzana w jednej iteracji pętli równoległej, a więc zapisywany jest unikalny element tablicy `result[i-m]`. Tablica `primeArray` jest w części równoległej wyłącznie odczytywana.

- **Wyścig:** Wyścig występuje wtedy, gdy co najmniej dwa wątki odwołują się jednocześnie do tej samej komórki pamięci, a przynajmniej jeden z nich wykonuje zapis. W wariancie k2 nie występuje wyścig na tablicy wynikowej, ponieważ każda iteracja pętli odpowiada innej liczbie `i` i zapisuje inny indeks `result[i-m]`. Nie ma też wyścigu na tablicy `primeArray`, ponieważ po przygotowaniu tablicy jest ona tylko odczytywana przez wątki.
- **False sharing:** W wariancie istnieje ryzyko false sharingu na tablicy `result`. Tablica ta składa się z 1-bajtowych wartości typu `bool`, dlatego wiele sąsiednich elementów mieści się w jednej 64-bajtowej linii pamięci podręcznej. Przy `schedule(dynamic)` domyślny rozmiar porcji wynosi jedną iterację, więc sąsiednie liczby mogą być sprawdzane przez różne wątki pracujące na różnych rdzeniach. Jeżeli takie wątki zapisują różne elementy tej samej linii cache, linia jest unieważniana w pozostałych rdzeniach i musi zostać ponownie pobrana. W tym wariancie narzut false sharingu jest jednak ograniczony, ponieważ zapis do `result` występuje co najwyżej raz dla danej liczby złożonej, a koszt obliczeń jest zdominowany przez operacje dzielenia modulo w pętli wewnętrznej.
- **Problemy efektywnościowe:** Głównym problemem efektywnościowym pozostaje zmienny koszt sprawdzania kolejnych liczb. Liczby złożone z małym dzielnikiem kończą się szybko, natomiast liczby pierwsze i liczby z dużym najmniejszym dzielnikiem wymagają większej liczby operacji modulo. Zastosowanie `schedule(dynamic)` poprawia zrównoważenie pracy, ale zwiększa narzut planisty OpenMP.
- **Synchronizacja:** Jawną synchronizację nie występuje. Synchronizacja pojawia się niejawnie na końcu konstrukcji `parallel for`.

Wariant k2_chunk: Równoległe dzielenie z dynamicznym przydziałem porcji

Zmodyfikowana wersja wariantu k2, w której dynamiczny przydział pojedynczych iteracji zastąpiono przydziałem porcji o rozmiarze `CHUNK_SIZE`.

```

1 #ifndef CHUNK_SIZE
2 #define CHUNK_SIZE 50
3 #endif
4
5 long long repeats = 6;
6
7 for (long long r = 0; r < repeats; r++) {
8     memset(result, true, range * sizeof(bool));
9     memset(primeArray, true, (limit + 1) * sizeof(bool));
10
11     #pragma omp parallel for schedule(dynamic, CHUNK_SIZE)
12     for (long long i = m; i <= n; i++) {
13         for (long long j = 2; j * j <= i; j++) {
14             if (primeArray[j] && i % j == 0) {
15                 result[i - m] = false;
16                 break;
17             }
18         }
19     }
20 }
```

Listing 3: Równoległa pętla dzielenia z porcjami iteracji (k2_chunk)

- **Znaczenie zmiennych:** `CHUNK_SIZE` określa liczbę kolejnych iteracji przekazywanych wątkowi w jednym przydziale pracy. W badanej wersji domyślna wartość wynosi 50.
- **Podział pracy i przydział zadań:** Podstawowym zadaniem pozostaje sprawdzenie jednej liczby, ale OpenMP przydziela zadania grupami po `CHUNK_SIZE` kolejnych liczb. Po zakończeniu jednej porcji wątek pobiera następną dostępną porcję.
- **Szeregowanie i zrównoważenie:** `schedule(dynamic, CHUNK_SIZE)` jest kompromisem między balansem pracy a narzutem planisty. Większa porcja zmniejsza liczbę przydziałów pracy i poprawia ciągłość dostępu do `result`, ale może pogorszyć balans. Mniejsza porcja lepiej równoważy zmienny koszt sprawdzania liczb, ale zwiększa narzut OpenMP.
- **Dyrektywy OpenMP:** Zastosowano dyrektywę `#pragma omp parallel for schedule(dynamic, CHUNK_SIZE)`. W porównaniu z k2 dodano sterowanie rozmiarem porcji, co pozwala badać wpływ parametru `CHUNK_SIZE` na efektywność obliczeń.
- **Problemy poprawnościowe:** Wariant pozostaje poprawny współbieżnie względem wariantu pierwotnego, każda liczba jest sprawdzana dokładnie raz w ramach jednego powtórzenia, a każda iteracja zapisuje inny element tablicy `result`.
- **Wyścig:** W wariancie k2_chunk nie występuje wyścig na danych wynikowych. Mimo że tablica `result` jest współdzielona, każda liczba `i` należy do dokładnie jednej iteracji pętli, a więc do dokładnie jednego

wątku. Wątek zapisuje tylko element `result[i-m]` odpowiadający aktualnie sprawdzanej liczbie. Tablica `primeArray` jest po inicjalizacji tylko odczytywana, dlatego również nie powoduje konfliktów zapisu.

- **False sharing:** Wariant `k2_chunk` nadal może powodować false sharing na tablicy `result`, ale jest na niego mniej podatny niż wariant `k2` z domyślnym `schedule(dynamic)`. Przydział porcji po `CHUNK_SIZE` kolejnych liczb sprawia, że jeden wątek przez pewien czas pracuje na ciągłym fragmencie tablicy wynikowej. Ogranicza to mieszanie sąsiednich indeksów pomiędzy wątkami. False sharing może jednak wystąpić na granicach porcji, szczególnie gdy `CHUNK_SIZE` nie jest dopasowany do rozmiaru linii cache. Dla `CHUNK_SIZE = 50` porcja obejmuje około 50 bajtów tablicy `bool`, czyli mniej niż typowa 64-bajtowa linia cache. W efekcie sąsiednie porcje mogą częściowo współdzielić tę samą linię pamięci podręcznej.
- **Problemy efektywnościowe:** Dobór `CHUNK_SIZE` wpływa jednocześnie na narzut planisty, balans pracy oraz lokalność zapisu. Mała porcja poprawia zrównoważenie, ale zwiększa liczbę przydziałów pracy i ryzyko false sharingu. Większa porcja zmniejsza narzut OpenMP oraz zapewnia bardziej ciągły zapis do `result`, lecz może pogorszyć balans, ponieważ koszt sprawdzania kolejnych liczb jest zmienny.
- **Synchronizacja:** Na końcu każdej pętli `parallel for` występuje niejawna bariera.

Wariant k3: Sekwencyjne wykreślanie wielokrotności

Wariant sekwencyjny oparty na sicie Eratostenesa. Zamiast dzielenia każdej liczby przez potencjalne dzielniki, kod wykreśla z tablicy `result` wielokrotności kolejnych liczb pierwszych.

```
1 long long limit = (long long)sqrt(n);
2 long long repeats = 50;
3
4 for (long long r = 0; r < repeats; r++) {
5     memset(result, true, range * sizeof(bool));
6     memset(primeArray, true, (limit + 1) * sizeof(bool));
7
8     for (long long i = 2; i * i * i * i <= n; i++) {
9         if (primeArray[i]) {
10             for (long long j = i * i; j * j <= n; j += i) {
11                 primeArray[j] = false;
12             }
13
14             for (long long i = 2; i <= limit; i++) {
15                 if (primeArray[i]) {
16                     long long firstMultiple = (m / i);
17                     if (firstMultiple <= 1) {
18                         firstMultiple = i + i;
19                     }
20                     else if (m % i) {
21                         firstMultiple = (firstMultiple * i) + i;
22                     }
23                     else {
24                         firstMultiple = (firstMultiple * i);
25                     }
26
27                     for (long long j = firstMultiple; j <= n; j += i) {
28                         result[j - m] = false;
29                     }
30                 }
31             }
32         }
33     }
```

Listing 4: Sekwencyjne wyznaczanie dzielników i wykreślanie wielokrotności (k3)

- **Znaczenie zmiennych:** Zmienne `m`, `n`, `range`, `result` oraz `primeArray` mają takie samo znaczenie jak w poprzednich wariantach. Zmienna `limit = sqrt(n)` określa największy potrzebny dzielnik. Zmienna `firstMultiple` wyznacza pierwszą wielokrotność liczby `i`, która znajduje się w badanym zakresie. Zmienna `repeats` służy do wielokrotnego wykonania obliczeń i uśrednienia czasu.
- **Przebieg przetwarzania:** Najpierw wyznaczana jest pomocnicza tablica `primeArray` dla liczb do `sqrt(n)`. Następnie kod przechodzi po kolejnych liczbach `i`, które pozostały oznaczone jako pierwsze, i wykreśla ich wielokrotności w tablicy `result`. Dla każdego dzielnika obliczana jest pierwsza wielokrotność należąca do zakresu $\langle m, n \rangle$, a następnie pętla z krokiem `i` ustawia odpowiednie elementy `result` na `false`.
- **Podział pracy i przydział zadań:** Wariant nie dzieli pracy pomiędzy wątki. Logicznym zadaniem jest wykreślenie wielokrotności jednej liczby pierwszej `i` z całego badanego zakresu. Koszt takiego zadania jest zmienny: dla małych wartości `i`, np. 2, 3, 5, liczba wykreślanych elementów jest bardzo duża, natomiast dla większych dzielników liczba wielokrotności maleje.
- **Szeregowanie i zrównoważenie:** Nie występuje szeregowanie OpenMP. Nierówny koszt kolejnych etapów wykreślania wpływa na czas działania, ale nie powoduje problemu zrównoważenia między wątkami, ponieważ obliczenia są wykonywane sekwencyjnie.

- **Dyrektywy OpenMP:** Wariant nie wykorzystuje dyrektyw zrównoleglających OpenMP.
- **Problemy poprawnościowe:** Dla badanych zakresów kod zwraca poprawne wyniki.
- **Problemy efektywnościowe:** Głównym ograniczeniem jest słaba lokalność dostępu do tablicy `result`. Dla każdego dzielnika `i` kod przechodzi przez potencjalnie duży fragment tablicy, a dla kolejnych dzielników wielokrotnie wraca do tych samych obszarów pamięci. Szczególnie dla dużego zakresu tablica `result` nie mieści się w pamięci podręcznej, co zwiększa koszt odczytów i zapisów. W porównaniu z wariantami dzielenia zmniejszona jest liczba kosztownych operacji modulo, ale zwiększa się znaczenie dostępu do pamięci.

Wariant k3a: Sekwencyjne wykreślanie blokowe

Zmodyfikowana wersja wariantu k3, w której wykreślanie wielokrotności odbywa się blokami. Celem zmiany jest poprawa lokalności dostępu do tablicy `result` przez ograniczenie aktualnie modyfikowanego fragmentu danych.

```

1 long long repeats = 120;
2 long long blockSize = 131072;
3 long long numberOfBlocks = range / blockSize;
4 if (range % blockSize != 0) {
5     numberOfBlocks++;
6 }
7
8 for (long long r = 0; r < repeats; r++) {
9     memset(result, true, range * sizeof(bool));
10    memset(primeArray, true, (limit + 1) * sizeof(bool));
11
12    for (long long i = 2; i * i * i * i <= n; i++) {
13        if (primeArray[i]) {
14            for (long long j = i * i; j * j <= n; j += i) {
15                primeArray[j] = false;
16            }
17        }
18
19        for (long long i = 0; i < numberOfBlocks; i++) {
20            long long low = m + i * blockSize;
21            long long high = m + i * blockSize + blockSize - 1;
22            if (high > n) {
23                high = n;
24            }
25
26            for (long long j = 2; j * j <= high; j++) {
27                if (primeArray[j]) {
28                    long long firstMultiple = (low / j);
29                    if (firstMultiple <= 1) {
30                        firstMultiple = j + j;
31                    }
32                    else if (low % j) {
33                        firstMultiple = (firstMultiple * j) + j;
34                    }
35                    else {
36                        firstMultiple = (firstMultiple * j);
37                    }
38
39                    for (long long k = firstMultiple; k <= high; k += j) {
40                        result[k - m] = false;
41                    }
42                }
43            }
44        }
45    }
46 }

```

Listing 5: Blokowe wykreślanie wielokrotności w zakresie lokalnym (k3a)

- **Znaczenie zmiennych:** Zmienna `blockSize` określa liczbę elementów zakresu przetwarzanych w jednym bloku. Zmienna `numberOfBlocks` określa liczbę bloków potrzebnych do pokrycia całego zakresu. Zmienne `low` i `high` wyznaczają granice aktualnie przetwarzanego bloku. Zmienna `k` oznacza kolejne wykreślane wielokrotności dzielnika `j` wewnątrz bieżącego bloku.
- **Przebieg przetwarzania:** Tak jak w wariantcie k3, najpierw wyznaczana jest tablica `primeArray`. Następnie zakres $\langle m, n \rangle$ dzielony jest na bloki. Dla każdego bloku kod przechodzi po liczbach pierwszych `j` nie większych niż $\sqrt{\text{high}}$ i wykreśla ich wielokrotności tylko w przedziale $\langle \text{low}, \text{high} \rangle$. Ostatni blok może być krótszy, dlatego jego koniec jest ograniczany warunkiem `if (high > n)`.
- **Podział pracy i przydział zadań:** Logicznym zadaniem jest przetworzenie jednego bloku danych - wewnątrz bloku wykreślane są wielokrotności kolejnych dzielników. Koszt bloków jest zbliżony, ponieważ większość bloków ma ten sam rozmiar, ale może różnić się liczbą wymaganych dzielników oraz liczbą wielokrotności przypadających do wykreślenia.

- **Problemy poprawnościowe:** Kod zachowuje poprawność wariantu k3, ponieważ każdy blok jest przetwarzany dla wszystkich potrzebnych dzielników pierwszych. Wyznaczenie `firstMultiple` względem początku bloku `low` zapewnia, że wykreślanie rozpoczyna się od pierwszej wielokrotności dzielnika należącej do aktualnego bloku.
- **Problemy efektywnościowe:** Względem k3 poprawiona została lokalność dostępu do tablicy `result`. W danym momencie modyfikowany jest tylko fragment o rozmiarze `blockSize`, dzięki czemu większa część operacji zapisu dotyczy ograniczonego obszaru pamięci. Zmniejsza to liczbę kosztownych odwołań do dalszych poziomów hierarchii pamięci oraz ogranicza liczbę nietrafień w pamięci podręcznej i TLB. Kosztem tej organizacji jest dodatkowa pętla po blokach oraz ponowne przechodzenie po dzielnikach dla każdego bloku.

Wariant k4: Równoległe wykreślanie z dynamicznym przydziałem dzielników

Zmodyfikowana wersja wariantu k3, w której zrównoleglono pętlę po dzielnikach pierwszych. Każdy wątek otrzymuje wybrane wartości `i` i wykreśla ich wielokrotności w całym badanym zakresie.

```

1 #pragma omp parallel for schedule(dynamic)
2 for (long long i = 2; i <= limit; i++) {
3     if (primeArray[i]) {
4         long long firstMultiple = (m / i);
5         if (firstMultiple <= 1) {
6             firstMultiple = i + i;
7         }
8         else if (m % i) {
9             firstMultiple = (firstMultiple * i) + i;
10        }
11        else {
12            firstMultiple = (firstMultiple * i);
13        }
14
15        for (long long j = firstMultiple; j <= n; j += i) {
16            result[j - m] = false;
17        }
18    }
19 }
```

Listing 6: Równoległe wykreślanie wielokrotności według dzielników (k4)

- **Znaczenie zmiennych:** Zmienne `m`, `n`, `range`, `result`, `primeArray` oraz `limit` mają takie samo znaczenie jak w wariancie k3. Zmienna `i` oznacza aktualnie przetwarzany dzielnik, a `firstMultiple` pierwszą jego wielokrotność należącą do badanego zakresu.
- **Przebieg przetwarzania:** Po sekwencyjnym przygotowaniu tablicy `primeArray` wykonywana jest równoległa pętla po potencjalnych dzielnikach `i`. Jeżeli `primeArray[i]` ma wartość `true`, wątek oblicza pierwszą wielokrotność liczby `i` w zakresie `<m,n>` i następnie wykreśla kolejne wielokrotności przez ustawienie odpowiednich elementów tablicy `result` na `false`.
- **Podział pracy i przydział zadań:** Zadaniem równoległym jest obsługa jednego dzielnika `i`, czyli wykreślenie jego wielokrotności w całym zakresie. Jest to podział funkcyjny, ponieważ wątki nie otrzymują osobnych fragmentów tablicy `result`, lecz różne funkcje/dzielniki do przetworzenia. Koszt zadań jest silnie zmienny: dla małych dzielników liczba zapisów jest bardzo duża, natomiast dla dużych dzielników liczba wielokrotności szybko maleje.
- **Szeregowanie i zrównoważenie:** Zastosowano `schedule(dynamic)`, aby ograniczyć niezrównoważenie wynikające z różnego kosztu kolejnych dzielników. Dynamiczny przydział pozwala wątkom pobierać kolejne wartości `i` po zakończeniu wcześniejszej pracy. Nie usuwa to jednak całkowicie problemu, ponieważ pojedyncze zadania dla małych dzielników, np. `i = 2` lub `i = 3`, są znacznie dłuższe niż większość pozostałych.
- **Dyrektywy OpenMP:** Dyrektywa `#pragma omp parallel for schedule(dynamic)` tworzy zespół wątków i rozdziela między nie iteracje pętli po dzielnikach. Indeks pętli `i` jest prywatny dla iteracji, natomiast tablice `primeArray` i `result` są współdzielone. Tablica `primeArray` jest tylko odczytywana, natomiast `result` jest modyfikowana przez wiele wątków.
- **Problemy poprawnościowe:** Wariant zwraca poprawny wynik liczbowy, ponieważ wszystkie konflikty zapisują tę samą wartość `false`. Nie jest jednak poprawny współbieżnie, ponieważ wiele wątków może jednocześnie zapisywać do tych samych elementów tablicy `result`.
- **Wyścig:** W wariancie k4 występuje wyścig, ponieważ ta sama liczba złożona może być wielokrotnością kilku różnych dzielników. Jeżeli różne wątki obsługują te dzielniki równoległe, mogą jednocześnie wykonać zapis do tego samego elementu.

- **False sharing:** Wariant charakteryzuje się wysokim ryzykiem wystąpienia false sharingu. Tablica `result` składa się z 1-bajtowych wartości typu `bool`, więc wiele kolejnych elementów mieści się w jednej 64-bajtowej linii cache. Ponieważ różne wątki obsługują różne dzielniki, ale zapisują w tym samym dużym zakresie `result`, mogą często modyfikować różne elementy tej samej linii cache. Dodatkowo zapis jest bezwarunkowy: nawet jeżeli dana liczba została już wcześniej wykreślona, kod ponownie zapisuje `false`. Zwiększa to liczbę unieważnień linii cache oraz ruch między pamięciami podręcznymi rdzeni.
- **Problemy efektywnościowe:** Główne problemy efektywnościowe to brak lokalności danych, duża liczba powtórných zapisów oraz niezrównoważony koszt zadań. Dla małych dzielników wątek przechodzi przez bardzo duży fragment tablicy, a inne wątki w tym samym czasie mogą modyfikować te same lub sąsiednie obszary. W efekcie koszt obliczeń jest silnie związany z obsługą zapisów do pamięci, mechanizmem spójności cache oraz możliwym false sharingiem. Wariant `k4` ma jednak prostszy przebieg sterowania niż `k4a`, ponieważ wewnętrzna pętla wykonuje bezwarunkowy zapis `result[j-m] = false`. Procesor nie musi w tym miejscu przewidywać wyniku dodatkowego warunku zależnego od danych, dlatego koszt błędnej predykcji rozgałęzień jest mniejszy niż w wariantcie `k4a`.
- **Synchronizacja:** Jawną synchronizację nie występuje. Na końcu konstrukcji `parallel for` występuje niejawną bariera, ale nie zabezpiecza ona dostępu do elementów tablicy `result` wewnątrz pętli.

Wariant `k4a`: Równoległe wykreślanie z warunkowym zapisem

Zmodyfikowana wersja wariantu `k4`, w której przed zapisem do tablicy `result` dodano sprawdzenie, czy dany element nadal ma wartość `true` w celu ograniczenia liczby powtórných zapisów do tablicy wynikowej.

```

1 #pragma omp parallel for schedule(dynamic)
2 for (long long i = 2; i <= limit; i++) {
3     if (primeArray[i]) {
4         long long firstMultiple = (m / i);
5         if (firstMultiple <= 1) {
6             firstMultiple = i + i;
7         }
8         else if (m % i) {
9             firstMultiple = (firstMultiple * i) + i;
10        }
11        else {
12            firstMultiple = (firstMultiple * i);
13        }
14
15        for (long long j = firstMultiple; j <= n; j += i) {
16            if (result[j - m]) {
17                result[j - m] = false;
18            }
19        }
20    }
21 }
```

Listing 7: Równoległe wykreślanie z warunkowym zapisem (`k4a`)

- **Znaczenie zmiennych:** Zmienne mają takie samo znaczenie jak w wariantcie `k4`.
- **Przebieg przetwarzania:** Przebieg jest taki sam jak w wariantcie `k4`, z tą różnicą, że przed zapisem dodano warunek `if (result[j-m])`. Jeżeli element jest już oznaczony jako złożony, zapis zostaje pominięty.
- **Podział pracy i przydział zadań:** Taki sam jak w wariantcie `k4`.
- **Szeregowanie i zrównoważenie:** Zastosowano dynamiczny przydział `schedule(dynamic)` ograniczający niezrównoważenie wynikające z różnych kosztów dzielników. Na procesorze hybrydowym dodatkowym czynnikiem może być różna wydajność rdzeni P i E, ponieważ ciężkie zadanie przydzielone do wolniejszego rdzenia może dłużej blokować zakończenie całej pętli.
- **Dyrektywy OpenMP:** Użyto dyrektywy `#pragma omp parallel for schedule(dynamic)`. Tablica `primeArray` jest współdzielona, ale tylko odczytywana. Tablica `result` jest współdzielona i zarówno odczytywana, jak i zapisywana przez wiele wątków.
- **Problemy poprawnościowe:** Wariant zwraca poprawny wynik liczbowy, ponieważ wszystkie zapisy do tablicy `result` ustawiają wartość `false`. Kod nie jest jednak poprawny współbieżnie, ponieważ warunek `if (result[j-m])` wykonuje niesynchronizowany odczyt elementu, który inny wątek może w tym samym czasie zapisywać.
- **Wyścig:** W wariantcie występują dwa rodzaje wyścigu. Pierwszy to `write-write` (taki sam jak w `k4`). Drugi to `read-write`, ponieważ jeden wątek może sprawdzać warunek `if (result[j-m])`, podczas gdy inny wątek zapisuje do tego samego elementu `false`. Z tego powodu wariant `k4a` jest formalnie bardziej problematyczny niż `k4`, mimo że często wykonuje mniej zapisów.

- **False sharing:** W wariancie nadal występują warunki sprzyjające false sharingowi, ponieważ wiele wątków operuje na tej samej tablicy `result`, a sąsiednie wartości typu `bool` współdzielą linie cache. W porównaniu z k4 liczba zapisów może być mniejsza, ponieważ elementy już oznaczone jako `false` nie są ponownie zapisywane. Nie usuwa to jednak problemu całkowicie, ponieważ każdy wątek nadal odczytuje elementy tablicy `result`, a część linii cache może być jednocześnie odczytywana i unieważniana przez zapisy innych wątków.
- **Problemy efektywnościowe:** Wariant zmniejsza liczbę powtórnych zapisów względem k4, co może ograniczyć koszt obsługi spójności cache. Ceną jest jednak dodatkowy odczyt `result[j-m]` oraz dodatkowa instrukcja warunkowa `if` wewnątrz najczęściej wykonywanej pętli. Procesor próbuje przewidywać wynik tego warunku i spekulacyjnie wykonywać jedną ze ścieżek programu. Jeżeli przewidywanie jest błędne, wykonana spekulacyjnie praca zostaje odrzucona, a dalsze wykonanie musi zostać wznowione od właściwej ścieżki. W wariancie k4 w tym miejscu pętli występuje bezwarunkowy zapis `result[j-m] = false`, dlatego nie ma dodatkowego rozgałęzienia zależnego od aktualnego stanu tablicy wynikowej. Z tego powodu ograniczenie liczby zapisów w k4a nie musi przełożyć się na krótszy czas wykonania: zysk pamięciowy może zostać zniwelowany przez koszt dodatkowych odczytów, instrukcji warunkowych, błędów predykcji i większej liczby wykonywanych instrukcji.
- **Synchronizacja:** Jawną synchronizację nie występuje. Na końcu pętli `parallel for` występuje tylko niejawną bariera. Brak synchronizacji wewnątrz pętli jest przyczyną wyścigów na tablicy `result`.

Wariant k5: Równoległe sito domenowe z lokalnością danych

Wariant równoległy oparty na sicie Eratostenesa z domenowym podziałem pracy. Zakres badanych liczb jest dzielony na bloki, a każdy wątek wykreśla wielokrotności liczb pierwszych wyłącznie w przydzielonym fragmencie tablicy `result`.

```

1 long long repeats = 350;
2 long long blockSize = 32768;
3 long long range = n - m + 1;
4 long long numberOfBlocks = (range + blockSize - 1) / blockSize;
5
6 for (long long r = 0; r < repeats; r++) {
7     memset(result, true, range * sizeof(bool));
8     memset(primeArray, true, (limit + 1) * sizeof(bool));
9
10    for (long long i = 2; i * i * i * i <= n; i++) {
11        if (primeArray[i]) {
12            for (long long j = i * i; j * j <= n; j += i) {
13                primeArray[j] = false;
14            }
15        }
16
17        #pragma omp parallel for schedule(dynamic)
18        for (long long i = 0; i < numberOfBlocks; i++) {
19            long long low = m + i * blockSize;
20            long long high = m + i * blockSize + blockSize - 1;
21            if (high > n) {
22                high = n;
23            }
24
25            for (long long j = 2; j * j <= high; j++) {
26                if (primeArray[j]) {
27                    long long firstMultiple = (low / j);
28                    if (firstMultiple <= 1) {
29                        firstMultiple = j + j;
30                    }
31                    else if (low % j) {
32                        firstMultiple = (firstMultiple * j) + j;
33                    }
34                    else {
35                        firstMultiple = (firstMultiple * j);
36                    }
37
38                    for (long long k = firstMultiple; k <= high; k += j) {
39                        result[k - m] = false;
40                    }
41                }
42            }
43        }
44    }
45 }

```

Listing 8: Równoległe wykreślanie bloków zakresu (k5)

- **Znaczenie zmiennych:** Zmiennie `m`, `n`, `range`, `result`, `primeArray` oraz `limit` mają takie samo znaczenie jak w wariantach sitowych. Zmienna `blockSize` określa rozmiar pojedynczego bloku, a `numberOfBlocks` liczbę bloków potrzebnych do pokrycia całego zakresu. Zmiennie `low` i `high` wyznaczają granice bloku przydzielonego danej iteracji pętli równoległej.
- **Przebieg przetwarzania:** Po przygotowaniu tablicy `primeArray` zakres `<m,n>` jest dzielony na bloki. Każda iteracja pętli równoległej przetwarza jeden blok. Wewnątrz bloku kod przechodzi po kolejnych dzielnikach pierwszych i wykreśla ich wielokrotności tylko w przedziale `<low,high>`.
- **Podział pracy i przydział zadań:** Zadaniem równoległym jest przetworzenie jednego bloku danych. Jest to podział domenowy, ponieważ wątki otrzymują rozłączne fragmenty tablicy `result`. Koszt zadań jest bardziej wyrównany niż w wariacie `k4`, ponieważ bloki mają zbliżony rozmiar. Nadal może się jednak różnić w zależności od wartości `high`, liczby potrzebnych dzielników oraz liczby wielokrotności w danym bloku.
- **Szeregowanie i zrównoważenie:** Zastosowano `schedule(dynamic)`, dzięki czemu wątki pobierają kolejne bloki po zakończeniu wcześniejszych. Pozwala to ograniczyć niezrównoważenie pracy, szczególnie na procesorze hybrydowym, gdzie rdzenie P i E mają różną wydajność. Narzut dynamicznego przydziału jest mniejszy niż w wariantach z bardzo drobnymi zadaniami, ponieważ pojedyncze zadanie obejmuje cały blok danych.
- **Dyrektywy OpenMP:** Dyrektywa `#pragma omp parallel for schedule(dynamic)` rozdziela bloki pomiędzy wątki. Indeks bloku `i` jest prywatny dla iteracji, natomiast `result` i `primeArray` są współdzielone. Tablica `primeArray` jest tylko odczytywana, a tablica `result` jest zapisywana przez wiele wątków, ale w rozłącznych zakresach indeksów.
- **Problemy poprawnościowe:** Wariant jest poprawny współbieżnie dla badanych zakresów, ponieważ każdy blok odpowiada innemu fragmentowi tablicy `result`. Każdy element wyniku należy do jednego bloku i jest zapisywany tylko przez wątek obsługujący ten blok.
- **Wyścig:** W wariacie `k5` nie występuje wyścig na tablicy wynikowej. Wątki mogą korzystać z tej samej tablicy `result`, ale zapisują rozłączne fragmenty wynikające z różnych wartości `low` i `high`. Tablica `primeArray` jest po przygotowaniu tylko odczytywana, dlatego również nie powoduje konfliktów zapisu.
- **False sharing:** Wariant znacząco ogranicza ryzyko false sharingu względem `k4` i `k4a`, ponieważ wątki zapisują całe, rozłączne bloki tablicy `result`. Potencjalny false sharing może wystąpić głównie na granicach sąsiednich bloków, gdy końcowe elementy jednego bloku i początkowe elementy następnego znajdują się w tej samej linii cache. Ponieważ jednak `blockSize` jest duży, koszt takiego zjawiska jest niewielki w porównaniu z liczbą zapisów wykonywanych wewnątrz bloków.
- **Problemy efektywnościowe:** Główną zaletą wariantu jest połączenie lokalności blokowej z bezpiecznym podziałem pracy. W porównaniu z `k4` ograniczona zostaje liczba konfliktów o te same linie cache, a w porównaniu z `k3a` bloki mogą być przetwarzane równolegle. Kosztem jest konieczność ponownego przechodzenia po dzielnikach dla każdego bloku oraz narzut dynamicznego przydziału bloków.
- **Synchronizacja:** Jawna synchronizacja nie występuje. Na końcu konstrukcji `parallel for` występuje niejawna bariera. Brak dodatkowej synchronizacji jest możliwy, ponieważ wątki zapisują rozłączne fragmenty tablicy wynikowej.

Wariant k5b: Równoległe sito domenowe z lokalnością danych i reprezentacją bitową

Dodatkowa modyfikacja wariantu `k5`, w której tablicę `bool` zastąpiono reprezentacją bitową. Celem jest zmniejszenie rozmiaru tablicy wynikowej i poprawa lokalności dostępu do danych.

```

1 #define SET_BIT(array, index) (array[(index) >> 3] |= (1 << ((index) & 7)))
2 #define CLEAR_BIT(array, index) (array[(index) >> 3] &= ~(1 << ((index) & 7)))
3 #define GET_BIT(array, index) ((array[(index) >> 3] & (1 << ((index) & 7))) != 0)
4
5 long long range = n - m + 1;
6 long long byteSize = (range + 7) / 8;
7 unsigned char* result = (unsigned char*)malloc(byteSize);
8 long long repeats = 1000;
9 long long blockSize = 131072;
10 long long numberOfBlocks = (range + blockSize - 1) / blockSize;
11
12 for (long long r = 0; r < repeats; r++) {
13     memset(result, 0xFF, byteSize);
14     memset(primeArray, true, (limit + 1) * sizeof(bool));
15
16     #pragma omp parallel for schedule(dynamic)
17     for (long long i = 0; i < numberOfBlocks; i++) {

```

```

18     long long low = m + i * blockSize;
19     long long high = m + i * blockSize + blockSize - 1;
20     if (high > n) {
21         high = n;
22     }
23
24     for (long long j = 2; j * j <= high; j++) {
25         if (primeArray[j]) {
26             long long firstMultiple = (low / j);
27             if (firstMultiple <= 1) {
28                 firstMultiple = j + j;
29             }
30             else if (low % j) {
31                 firstMultiple = (firstMultiple * j) + j;
32             }
33             else {
34                 firstMultiple = (firstMultiple * j);
35             }
36
37             for (long long k = firstMultiple; k <= high; k += j) {
38                 CLEAR_BIT(result, k - m);
39             }
40         }
41     }

```

Listing 9: Reprezentacja bitowa i równoległe wykreślanie bloków (k5b)

- **Znaczenie zmiennych:** Zmienna `byteSize` określa liczbę bajtów potrzebnych do zapisania range wartości logicznych w postaci bitowej. Makro `CLEAR_BIT` zeruje bit odpowiadający liczbie złożonej, a `GET_BIT` służy do odczytu wyniku podczas końcowego zliczania liczb pierwszych. Wartość `0xFF` w funkcji `memset` ustawia początkowo wszystkie bity na 1, czyli traktuje wszystkie liczby jako potencjalnie pierwsze.
- **Przebieg przetwarzania:** Przebieg jest taki sam jak w wariancie `k5`: zakres jest dzielony na bloki, a każdy blok jest przetwarzany przez jeden wątek. Różnica polega na sposobie zapisu wyniku. Zamiast ustawiać `result[k-m] = false`, kod zeruje pojedynczy bit odpowiadający liczbie `k`.
- **Podział pracy i przydział zadań:** Podział pracy pozostaje domenowy. Zadaniem jest przetworzenie jednego bloku liczb, ale w pamięci wynikowej jeden bajt przechowuje stan ośmiu kolejnych liczb. W badanej wersji `blockSize` jest większy niż w `k5`, co zmniejsza liczbę zadań i narzut planisty, ale może ograniczać elastyczność dynamicznego równoważenia pracy.
- **Szeregowanie i zrównoważenie:** Zastosowano `schedule(dynamic)`. Dynamiczny przydział bloków nadal pomaga w równoważeniu pracy, natomiast większy rozmiar bloku oznacza mniejszą liczbę przydziałów pracy. Jest to kompromis między narzutem OpenMP a możliwością równomiernego obciążenia rdzeni.
- **Dyrektywy OpenMP:** Użyto dyrektywy `#pragma omp parallel for schedule(dynamic)`. Tablica `primeArray` jest współdzielona i tylko odczytywana. Tablica `result` jest współdzielona, ale każdy wątek modyfikuje bity należące do swojego bloku.
- **Problemy poprawnościowe:** Wariant zachowuje poprawność dla badanych zakresów. Ponieważ tablica wynikowa jest spakowana bitowo, operacja zmiany pojedynczego bitu modyfikuje cały bajt. Dlatego ważne jest, aby granice bloków nie powodowały sytuacji, w której dwa wątki modyfikują różne bity tego samego bajtu. W badanej konfiguracji `blockSize` jest wielokrotnością 8, więc granice bloków wypadają na granicach bajtów.
- **Wyścigi:** Przy zastosowanym rozmiarze bloku wyścig na tablicy wynikowej nie występuje, ponieważ każdy bajt reprezentujący fragment wyniku należy do jednego bloku. Gdyby `blockSize` nie był wielokrotnością 8, dwa sąsiednie bloki mogłyby współdzielić jeden bajt tablicy bitowej, co mogłoby prowadzić do wyścigu typu `read-modify-write` podczas wykonywania makra `CLEAR_BIT`.
- **False sharing:** Reprezentacja bitowa zmniejsza rozmiar tablicy wynikowej około ośmiokrotnie, co ogranicza liczbę linii cache potrzebnych do przechowania wyniku. Wariant nadal może powodować `false sharing` na granicach bloków, jeżeli bajty należące do sąsiednich bloków znajdują się w tej samej linii cache. Ryzyko to jest jednak ograniczone przez duży rozmiar bloku oraz przez fakt, że wątek wykonuje większość zapisów wewnątrz własnego fragmentu danych.
- **Problemy efektywnościowe:** Główną zaletą wariantu jest mniejszy rozmiar zbioru danych. Dzięki temu mniej danych musi być przenoszonych między poziomami pamięci, a tablica wynikowa lepiej wykorzystuje linie cache. Kosztem są dodatkowe operacje bitowe oraz operacje typu `odczyt-modyfikacja-zapis` wykonywane podczas zerowania bitów. W praktyce wariant może być szybszy od `k5`, jeżeli zysk z mniejszego rozmiaru danych przewyższa koszt dodatkowych operacji bitowych.
- **Synchronizacja:** Jawną synchronizację nie występuje. Na końcu pętli `parallel for` występuje niejawną bariera. Brak synchronizacji wewnątrz pętli jest bezpieczny przy założeniu, że bloki nie współdzielą tych

samych bajtów tablicy bitowej.

Wariant k5c: Równoległe sito domenowe liczb nieparzystych z lokalnością danych

Dodatkowa modyfikacja wariantu k5, w której z tablicy wynikowej usunięto liczby parzyste. Tablica `result` przechowuje wyłącznie informacje o liczbach nieparzystych, a liczba 2 jest doliczana osobno, jeżeli należy do badanego zakresu.

```
1 long long m_real = M, n = N;
2 long long m = (m_real % 2 == 0) ? m_real + 1 : m_real;
3
4 long long primeCount = (m_real <= 2 && n >= 2) ? 1 : 0;
5
6 long long rangeOdds = (n - m) / 2 + 1;
7 bool* result = (bool*)malloc(rangeOdds * sizeof(bool));
8
9 long long repeats = 1000;
10 long long blockSize = 32768;
11 long long numberOfBlocks = rangeOdds / blockSize;
12 if (rangeOdds % blockSize != 0) {
13     numberOfBlocks++;
14 }
15
16 for (long long r = 0; r < repeats; r++) {
17     memset(result, true, rangeOdds * sizeof(bool));
18     memset(primeArray, true, (limit + 1) * sizeof(bool));
19
20     #pragma omp parallel for schedule(dynamic)
21     for (long long i = 0; i < numberOfBlocks; i++) {
22         long long lowIndex = i * blockSize;
23         long long highIndex = lowIndex + blockSize - 1;
24         if (highIndex >= rangeOdds) {
25             highIndex = rangeOdds - 1;
26         }
27
28         long long low = m + lowIndex * 2;
29         long long high = m + highIndex * 2;
30
31         for (long long j = 3; j * j <= high; j += 2) {
32             if (primeArray[j]) {
33                 long long firstMultiple = (low / j);
34                 if (firstMultiple < j) {
35                     firstMultiple = j;
36                 }
37
38                 long long start = firstMultiple * j;
39                 if (start < low) {
40                     start += j;
41                 }
42                 if (start % 2 == 0) {
43                     start += j;
44                 }
45
46                 long long step = j * 2;
47
48                 for (long long k = start; k <= high; k += step) {
49                     result[(k - m) / 2] = false;
50                 }
51             }
52         }
53     }
```

Listing 10: Blokowe wykreślanie tylko liczb nieparzystych (k5c)

- **Znaczenie zmiennych:** Zmienna `m_real` oznacza rzeczywisty początek badanego zakresu, natomiast `m` jest przesunięte do pierwszej liczby nieparzystej w tym zakresie. Zmienna `rangeOdds` określa liczbę nieparzystych wartości przechowywanych w tablicy `result`. Indeks `result[i]` odpowiada liczbie `m + 2*i`. Zmienna `primeCount` jest inicjalizowana wartością 1 tylko wtedy, gdy liczba 2 należy do badanego zakresu.
- **Przebieg przetwarzania:** Kod pomija wszystkie liczby parzyste, ponieważ poza liczbą 2 nie mogą one być pierwsze. Zakres nieparzystych liczb jest dzielony na bloki indeksów. Dla każdego bloku wyznaczane są rzeczywiste wartości `low` i `high`, a następnie wykreślane są tylko nieparzyste wielokrotności nieparzystych dzielników. Z tego powodu pętla po dzielnikach zaczyna się od 3 i zwiększa `j` o 2, a pętla po wielokrotnościach używa kroku `step = j * 2`.
- **Podział pracy i przydział zadań:** Zadaniem równoległym jest przetworzenie jednego bloku indeksów

tablicy `result`. Podział pozostaje domenowy, ale domena jest mniejsza niż w `k5`, ponieważ obejmuje tylko liczby nieparzyste. Każdy wątek pracuje na rozłącznym fragmencie tablicy wynikowej.

- **Szeregowanie i zrównoważenie:** Zastosowano `schedule(dynamic)`. Dynamiczny przydział bloków pozwala zmniejszyć niezrównoważenie wynikające z różnego kosztu bloków. Ponieważ zakres danych jest mniejszy niż w `k5`, pojedyncze bloki reprezentują większy fragment rzeczywistego zakresu liczb przy tej samej liczbie elementów tablicy.
- **Dyrektywy OpenMP:** Dyrektywa `#pragma omp parallel for schedule(dynamic)` rozdziela bloki liczb nieparzystych pomiędzy wątki. Tablica `primeArray` jest współdzielona i tylko odczytywana. Tablica `result` jest współdzielona, ale każdy wątek zapisuje inny zakres indeksów.
- **Problemy poprawnościowe:** Wariant jest poprawny dla badanych zakresów, ponieważ liczby parzyste są pomijane świadomie, a liczba 2 jest doliczana osobno tylko wtedy, gdy należy do zakresu. Wyrażenie `result[(k-m)/2]` poprawnie przelicza nieparzystą liczbę `k` na indeks w tablicy przechowującej wyłącznie liczby nieparzyste.
- **Wyścig:** W wariantcie `k5c` nie występuje wyścig na tablicy wynikowej. Każdy blok odpowiada innemu zakresowi indeksów tablicy `result`, a więc element `result[(k-m)/2]` należy do dokładnie jednego bloku. Tablica `primeArray` jest tylko odczytywana w części równoległej.
- **False sharing:** Wariant ogranicza ryzyko false sharingu względem `k5`, ponieważ liczba elementów tablicy wynikowej jest mniejsza i wykonywanych jest mniej zapisów. Nadal może wystąpić false sharing na granicach bloków, jeżeli sąsiednie bloki zapisują elementy znajdujące się w tej samej linii cache. Ze względu na duży rozmiar bloków zjawisko to powinno mieć jednak ograniczony wpływ na czas wykonania.
- **Problemy efektywnościowe:** Główną zaletą wariantu jest zmniejszenie liczby przetwarzanych elementów prawie dwukrotnie. Kod nie zapisuje wyników dla liczb parzystych i nie wykreśla parzystych wielokrotności, co zmniejsza liczbę operacji oraz rozmiar aktywnie używanej tablicy. Kosztem jest bardziej złożone przeliczanie indeksów oraz dodatkowa logika wyznaczania pierwszej nieparzystej wielokrotności w bloku.
- **Synchronizacja:** Jawna synchronizacja nie występuje. Na końcu pętli `parallel for` występuje niejawna bariera. Brak synchronizacji wewnątrz pętli jest możliwy dzięki domenowemu podziałowi tablicy wynikowej.

Uzasadnienie kluczowych parametrów algorytmów

Określenie efektywnego rozmiaru bloku danych

Zgodnie z parametrami procesora Intel i5-12500H, rdzenie wyposażone są w bardzo szybką pamięć podręczną L1 (48 KB na dane dla P-Core, 32 KB dla E-Core) oraz pamięć L2 (1.25 MB dla P-Core, 2 MB współdzielone dla klastra E-Core).

- **Zbiór roboczy:** Dla górnego zakresu badanego przedziału (10^8), tablica dzielników bazowych `primeArray` o rozmiarze $\sqrt{10^8} \approx 10000$ elementów typu `bool` zajmuje około 10 KB.
- **Optymalizacja pod Cache L1 (k5):** W celu zapewnienia maksymalnej lokalności danych dla sita równoległego, rozmiar bloku ustalono na `blockSize = 32768` (32 KB). Sumaryczny rozmiar danych przetwarzanych w jednym momencie (32 KB + 10 KB) wynosi 42 KB, co pozwala na całkowite zmieszczenie danych w pamięci L1 rdzeni P-Core oraz minimalizuje narzuty na rdzeniach E-Core.
- **Optymalizacja pod Cache L2 (k3a):** Dla sekwencyjnego sita blokowego przetestowano większy rozmiar bloku `blockSize = 131072` (128 KB). Pozwala on na efektywne wykorzystanie pojemnej pamięci L2, redukując częstotliwość zmian kontekstu wewnętrznej pętli.

Analiza sposobów szeregowania pętli

W wariantach zrównoleglonych (`k2`, `k4`, `k4a`, `k5`) kluczowym wyzwaniem jest skrajne zróżnicowanie czasu wykonania poszczególnych iteracji pętli. Na przykład wykreślanie wielokrotności liczby 2 wymaga bez porównania większej liczby operacji niż wykreślanie wielokrotności liczby 9973.

- **Szeregowanie statyczne (`schedule(static)`):** Zastosowanie domyślnego, statycznego podziału iteracji prowadzi do drastycznego niezrównoważenia obciążenia. Wątki, które otrzymały najmniejsze liczby pierwsze (lub zakresy początkowe), pracują znacznie dłużej, podczas gdy pozostałe wątki oczekują bezczynnie na zakończenie bloku `parallel`. Jest to szczególnie widoczne w hybrydowej architekturze procesora, gdzie rdzenie E-Core naturalnie wykonują zadania wolniej niż P-Core.
- **Szeregowanie dynamiczne (`schedule(dynamic)`):** W zaimplementowanych kodach wybrano szeregowanie dynamiczne, które przydziela kolejne pakiety zadań wątkom zwalniającym się po zakończeniu po-

przedniej pracy. Metoda ta naturalnie równoważy obciążenie zarówno pomiędzy nierównomiernymi matematycznie iteracjami algorytmu, jak i asymetryczną wydajnością rdzeni procesora.

Punkt 3: Prezentacja wyników i omówienie przebiegu eksperymentu

a) Charakterystyka testowanych instancji i wariantów

W eksperymencie przetestowano wszystkie podstawowe warianty: k1, k2, k2_chunk, k3, k3a, k4, k4a, k5, k5b, k5c oraz warianty stanowiące rozszerzenie w ramach punktu 3f k2_chunk, k5b i k5c.

Pomiary wykonano dla trzech zakresów:

Tabela 1: Wartości referencyjne liczby liczb pierwszych dla badanych zakresów.

Zakres	Liczba liczb pierwszych
<2, 100000000>	5761455
<2, 50000000>	3001134
<50000000, 100000000>	2760321

Dla każdego zakresu sprawdzono poprawność wyniku przez porównanie liczby znalezionych liczb pierwszych z wartością referencyjną dla każdej wersji kodu. Pomiary wykonano przy jednakowym sposobie kompilacji kodów, z włączoną optymalizacją -O3 oraz obsługą OpenMP. Dla wariantów, których pojedyncze wykonanie było bardzo krótkie, obliczenia powtarzano wielokrotnie w jednym uruchomieniu programu. W tabelach wynikowych czas jednego wykonania obliczono jako $T_1 = T_{VTune}/repeats$, gdzie T_{VTune} oznacza czas Elapsed Time z raportu VTune, a $repeats$ oznacza liczbę powtórzeń wykonaną w programie.

Tabela 2: Liczba powtórzeń użyta do uśrednienia czasu jednego wykonania.

Wariant	k1	k2	k2_chunk	k3	k3a	k4	k4a	k5	k5b	k5c
Liczba powtórzeń	1	1	6	50	120	50	50	350	1000	1000

b) Zasada pracy oprogramowania Intel VTune Profiler

Do analizy wykorzystano eksperyment *Microarchitecture Exploration* w Intel VTune Profiler. Narzędzie zbiera informacje na podstawie sprzętowych liczników zdarzeń procesora, tzw. PMU – *Performance Monitoring Unit*. Dzięki temu możliwe jest określenie nie tylko czasu wykonania programu, ale także przyczyn ograniczających wydajność, np. oczekiwania na dane z pamięci, ograniczeń jednostek wykonawczych lub problemów z pobieraniem instrukcji.

VTune nie mierzy każdej instrukcji osobno, lecz wykorzystuje próbkowanie zdarzeń sprzętowych. W określonych odstępach zbierane są informacje o stanie wykonania programu i zdarzeniach zachodzących w procesorze. Następnie narzędzie przypisuje zebrane próbki do funkcji, fragmentów kodu oraz kategorii mikroarchitektonicznych. Wykorzystany tryb *Microarchitecture Exploration* opiera się na analizie typu *Top-Down Microarchitecture Analysis*, która klasyfikuje wykorzystanie potoku procesora na kategorie takie jak *Retiring*, *Front-End Bound* i *Back-End Bound*.

W tabelach wykorzystano następujące metryki:

- **Elapsed Time** – rzeczywisty czas wykonania programu mierzony od rozpoczęcia do zakończenia analizowanego przebiegu. Dla wariantów wykonywanych wielokrotnie wartość tę podzielono przez liczbę powtórzeń, aby otrzymać średni czas pojedynczego wykonania.
- **Instructions Retired** – liczba instrukcji zakończonych poprawnie przez procesor. Wysoka liczba instrukcji oznacza większą ilość pracy wykonanej przez procesor, ale nie musi bezpośrednio oznaczać dłuższego czasu, ponieważ znaczenie ma także rodzaj instrukcji, dostęp do pamięci i stopień równoległości.
- **Clockticks** – liczba cykli zegara zużytych podczas wykonania programu. W połączeniu z liczbą instrukcji pozwala ocenić, ile cykli średnio przypadało na wykonanie jednej instrukcji.
- **Retiring** – procent czasu pracy procesora wykorzystanego na instrukcje zakończone poprawnie. Wysoka wartość oznacza, że procesor dużą część czasu wykonywał użyteczną pracę. Niska wartość wskazuje, że znaczna część potencjału procesora była tracona na przestoje.
- **Front-End Bound** – udział ograniczeń wynikających z części procesora odpowiedzialnej za pobieranie i dekodowanie instrukcji. Wysoka wartość może oznaczać problemy z dostarczaniem instrukcji do dalszych etapów potoku, np. przez nietrafienia w cache instrukcji lub złożony przebieg sterowania programu.

- **Back-End Bound** – udział ograniczeń wynikających z części wykonawczej procesora. Oznacza sytuację, w których instrukcje są już dostarczone, ale nie mogą zostać wykonane z powodu oczekiwania na dane lub zajętości jednostek wykonawczych.
- **Memory Bound** – część ograniczenia *Back-End Bound* związana z dostępem do pamięci. Wysoka wartość oznacza, że program często czeka na dane z pamięci podręcznej niższego poziomu, pamięci głównej albo traci czas przez nietrafienia w cache i TLB. Jest to szczególnie ważne przy analizie wariantów sitowych, które intensywnie zapisują tablicę `result`.
- **Core Bound** – część ograniczenia *Back-End Bound* wynikająca z zajętości jednostek wykonawczych rdzenia. Wysoka wartość oznacza, że problemem nie jest głównie pamięć, lecz ograniczona przepustowość rdzenia dla wykonywanych instrukcji, np. operacji arytmetycznych, bitowych lub warunkowych.
- **Effective Physical Core Utilization** – efektywne wykorzystanie rdzeni fizycznych procesora. Metryka pokazuje, w jakim stopniu program faktycznie angażował dostępne rdzenie. Sama wysoka wartość tej metryki nie gwarantuje krótkiego czasu wykonania, ponieważ program może dobrze zajmować rdzenie, ale jednocześnie tracić dużo czasu na oczekiwanie na pamięć.

Ponieważ procesor Intel Core i5-12500H ma architekturę hybrydową, VTune raportuje część metryk oddzielnie dla rdzeni P-core i E-core. Rdzenie P-core są rdzeniami wydajnościowymi, natomiast rdzenie E-core są rdzeniami energooszczędnymi o innej charakterystyce wykonania.

c) Zbiorcze zestawienia wyników eksperymentów

Wartości przyspieszenia (S_p) i efektywności (E_p) wyznaczono względem najlepszego wariantu sekwencyjnego (**k3a**), dla $p = 12$ rdzeni fizycznych. Tabele prezentują uśrednione statystyki z rozbiciem na rdzenie P-Core (P) i E-Core (E). Jednostką dla instrukcji i cykli jest Giga (10^9), liczba przetestowanych liczb wyrażana jest w milionach liczb na sekundę (M/s), kolumna E_p oznacza efektywność przetwarzania równoległego, obliczoną jako $E_p = S_p/12$, gdzie 12 oznacza liczbę fizycznych rdzeni procesora. Dla wariantów **k5b** i **k5c** wartości S_p i E_p należy interpretować jako efekt łączny zrównoleglenia oraz zmiany reprezentacji danych lub ograniczenia dziedziny obliczeń.

Tabela 3: Szczegółowe zestawienie wydajności dla pełnej instancji (zakres $\langle 2, 10^8 \rangle$)

Wariant	Typ	Czas [s]	Instr. [G]	Clock. [G]	M/s	S_p	E_p	Retir.	Front.B	Back.B	Mem.B	Core.B	Util.
k1 (Seq)	P	66,695	350,07	260,74	1,50	0,00	–	23,0%	33,3%	6,7%	0,1%	6,6%	7,7%
	E		33,70	17,12				41,1%	26,0%	1,8%	0,8%	0,9%	
k2 (Par)	P	5,866	191,52	155,06	17,05	0,02	0,00	41,3%	33,5%	4,7%	0,9%	3,9%	71,2%
	E		219,51	114,00				40,2%	25,3%	2,4%	1,3%	1,1%	
k2_chunk (C=50)	P	5,846	165,43	153,06	17,11	0,02	0,00	36,4%	33,5%	4,9%	0,9%	4,0%	76,3%
	E		219,19	113,68				40,4%	23,8%	1,9%	0,5%	1,4%	
k3 (Seq)	P	0,6308	0,54	2,74	158,52	0,17	–	3,5%	0,4%	96,3%	95,9%	0,4%	8,2%
	E		0,00	0,00				0,0%	5,8%	46,3%	45,3%	1,0%	
k3a (Seq)	P	0,1048	0,61	0,45	954,12	1,00	–	21,4%	1,9%	71,8%	70,1%	1,6%	8,1%
	E		0,00	0,00				0,0%	0,0%	28,9%	0,0%	28,9%	
k4 (Par)	P	0,2194	0,40	5,39	455,83	0,48	0,04	2,7%	3,5%	94,7%	88,5%	6,2%	59,1%
	E		0,56	3,86				3,1%	2,4%	92,0%	61,2%	30,8%	
k4a (Par)	P	0,2204	0,88	5,43	453,68	0,48	0,04	6,4%	15,9%	57,2%	43,5%	13,7%	59,8%
	E		0,93	3,98				5,9%	16,7%	53,5%	53,5%	0,0%	
k5 (Par)	P	0,0234	0,67	0,60	4267,77	4,47	0,37	39,1%	18,3%	31,0%	8,4%	22,6%	54,4%
	E		0,42	0,41				26,0%	11,4%	52,7%	33,3%	19,4%	
k5b (Bit)	P	0,0122	0,45	0,30	8218,28	8,61	0,72	70,5%	14,2%	10,1%	1,6%	8,5%	69,7%
	E		0,56	0,22				51,2%	7,1%	34,4%	2,3%	32,1%	
k5c (Odd)	P	0,0156	0,59	0,38	6406,56	6,71	0,56	50,5%	11,9%	27,0%	3,2%	23,7%	59,2%
	E		0,56	0,28				44,5%	10,4%	34,5%	9,7%	24,8%	

Tabela 4: Szczegółowe zestawienie wydajności dla pełnej instancji uruchomionej tylko na rdzeniach P-core (zakres $\langle 2, 10^8 \rangle$)

Wariant	Typ	Czas [s]	Instr. [G]	Clock. [G]	M/s	S_p	E_p	Retir.	Front.B	Back.B	Mem.B	Core.B	Util.
k1 (Seq)	P	113,712	384,27	297,27	0,88	0,00	–	23,5%	34,0%	6,8%	0,4%	6,4%	21,6%
k2 (Par)	P	12,233	411,99	340,87	8,17	0,01	0,00	39,9%	32,5%	4,3%	0,6%	3,7%	86,4%
k2_chunk	P	12,239	384,48	349,99	8,17	0,01	0,00	37,1%	30,8%	4,9%	0,8%	4,1%	92,6%
k3 (Seq)	P	0,6013	0,54	2,58	166,31	0,21	–	3,4%	0,5%	97,1%	94,1%	3,0%	23,9%
k3a (Seq)	P	0,1272	0,61	0,50	786,06	1,00	–	29,7%	7,0%	63,9%	62,7%	1,2%	18,2%
k4 (Par)	P	0,2929	0,76	7,49	341,41	0,43	0,11	3,4%	4,1%	88,9%	87,9%	1,0%	75,9%
k4a (Par)	P	0,2920	1,39	7,66	342,44	0,44	0,11	7,3%	19,9%	45,8%	38,6%	7,2%	82,3%
k5 (Par)	P	0,0282	1,00	0,76	3550,06	4,52	1,13	43,4%	20,7%	20,6%	12,9%	7,7%	87,0%
k5b (Bit)	P	0,0244	0,99	0,63	4095,17	5,21	1,30	74,8%	14,3%	5,4%	1,0%	4,3%	84,8%
k5c (Odd)	P	0,0221	1,09	0,58	4527,55	5,76	1,44	62,8%	11,3%	14,4%	4,4%	10,0%	84,8%

Porównanie uruchomienia na rdzeniach P-core z uruchomieniem na całym procesorze

Porównanie tabel 3 i 4 pokazuje wpływ wykorzystania rdzeni E-core na czas wykonania badanych wariantów. W uruchomieniu na całym procesorze dostępne są zarówno rdzenie P-core, jak i E-core, natomiast w eksperymencie P-core wykorzystane są wyłącznie cztery wydajne rdzenie.

Największa różnica widoczna jest w wariantach dzielenia. Wariant k2 na całym procesorze wykonuje się w czasie 5,866 s, natomiast przy ograniczeniu do samych rdzeni P-core w czasie 12,233 s. Podobnie k2_chunk zmienia czas z 5,846 s na 12,239 s. Oznacza to, że dla dużej liczby niezależnych iteracji dodatkowe rdzenie E-core wyraźnie zwiększają całkowitą przepustowość.

Warianty k4 i k4a również są szybsze przy użyciu całego procesora, ale różnica jest mniejsza: dla k4 czas zmienia się z 0,2929 s na 0,2194 s, a dla k4a z 0,2920 s na 0,2204 s. Wynika to z faktu, że te warianty są ograniczane głównie przez współbieżne zapisy do tablicy result, spójność cache oraz false sharing, więc samo zwiększenie liczby rdzeni nie usuwa głównego wąskiego gardła.

W wariantach domenowych różnica zależy od charakteru modyfikacji algorytmu. k5 przy użyciu całego procesora osiąga 0,0234 s, a na samych P-core 0,0282 s. Wariant k5b jest najszybszy przy użyciu całego procesora (0,0122 s), natomiast przy ograniczeniu do samych P-core najlepszy wynik uzyskuje k5c (0,0221 s). Wskazuje to, że k5b szczególnie dobrze korzysta z większej liczby rdzeni i zmniejszonego ruchu pamięciowego, natomiast przy mniejszej liczbie rdzeni większe znaczenie ma prostszy schemat przetwarzania liczb nieparzystych w k5c.

Tabela 5: Szczegółowe zestawienie wydajności dla dolnej połowy instancji (zakres $\langle 2, 5 \cdot 10^7 \rangle$)

Wariant	Typ	Czas [s]	Instr. [G]	Clock. [G]	M/s	S_p	E_p	Retir.	Front.B	Back.B	Mem.B	Core.B	Util.
k1 (Seq)	P	24,548	143,65	107,04	2,04	0,00	–	23,8%	34,3%	6,9%	0,1%	6,8%	8,1%
	E		0,02	0,02				11,6%	11,6%	11,6%	–	–	
k2 (Par)	P	2,232	74,46	57,17	22,40	0,02	0,00	41,9%	32,6%	3,1%	0,3%	2,8%	62,1%
	E		82,80	38,61				45,2%	24,3%	3,6%	2,4%	1,2%	
k2_chunk (C=50)	P	2,080	64,75	55,49	24,03	0,02	0,00	39,4%	33,0%	5,0%	0,9%	4,0%	71,4%
	E		79,39	37,81				44,3%	23,1%	2,6%	0,5%	2,1%	
k3 (Seq)	P	0,2841	0,26	1,23	176,02	0,18	–	4,7%	0,4%	97,4%	96,7%	0,6%	8,1%
	E		0,00	0,00				0,0%	0,0%	0,0%	0,0%	0,0%	
k3a (Seq)	P	0,0508	0,29	0,22	984,41	1,00	–	23,0%	1,5%	77,8%	75,2%	2,6%	7,9%
	E		0,00	0,00				28,9%	0,0%	86,8%	28,9%	57,9%	
k4 (Par)	P	0,0859	0,22	2,03	582,21	0,59	0,05	4,2%	4,6%	89,5%	81,5%	8,0%	54,9%
	E		0,24	1,37				4,2%	3,2%	92,4%	57,5%	34,8%	
k4a (Par)	P	0,0984	0,48	2,45	508,03	0,52	0,04	7,8%	18,5%	50,2%	32,4%	17,8%	57,0%
	E		0,52	1,78				7,1%	19,1%	44,9%	44,9%	0,0%	
k5 (Par)	P	0,0115	0,31	0,29	4363,00	4,43	0,37	37,2%	19,5%	36,4%	9,3%	27,2%	51,5%
	E		0,19	0,20				23,5%	12,3%	56,5%	30,0%	26,5%	
k5b (Bit)	P	0,0058	0,22	0,15	8629,62	8,77	0,73	71,4%	13,9%	10,2%	1,3%	8,8%	66,7%
	E		0,26	0,10				52,2%	6,7%	35,3%	2,8%	32,5%	
k5c (Odd)	P	0,0068	0,27	0,17	7356,19	7,47	0,62	51,0%	12,1%	29,1%	3,4%	25,7%	57,8%
	E		0,26	0,12				44,9%	11,0%	33,4%	10,0%	23,4%	

Tabela 6: Szczegółowe zestawienie wydajności dla górnej połowy instancji (zakres $\langle 5 \cdot 10^7, 10^8 \rangle$)

Wariant	Typ	Czas [s]	Instr. [G]	Clock. [G]	M/s	S_p	E_p	Retir.	Front.B	Back.B	Mem.B	Core.B	Util.
k1 (Seq)	P	55,337	13,79	10,37	0,90	0,00	–	22,0%	34,8%	6,2%	0,4%	5,8%	7,4%
	E		226,63	115,57				40,4%	25,3%	1,8%	0,7%	1,0%	
k2 (Par)	P	4,080	119,43	103,15	12,25	0,03	0,00	37,0%	32,7%	4,7%	0,6%	4,1%	67,2%
	E		134,70	73,32				38,4%	26,0%	2,7%	1,6%	1,1%	
k2_chunk (C=50)	P	3,481	97,94	88,65	14,37	0,03	0,00	36,8%	34,4%	4,8%	0,8%	3,9%	73,5%
	E		142,69	65,04				45,9%	24,7%	2,9%	0,8%	2,1%	
k3 (Seq)	P	0,3035	0,28	1,29	164,72	0,40	–	4,3%	0,4%	95,3%	96,9%	0,0%	7,7%
	E		0,00	0,00				2,5%	2,5%	100,0%	0,0%	100,0%	
k3a (Seq)	P	0,1209	0,12	0,09	413,56	1,00	–	22,7%	2,9%	71,8%	63,4%	8,4%	6,2%
	E		0,22	0,28				16,5%	1,8%	75,7%	63,6%	12,1%	
k4 (Par)	P	0,0888	0,17	2,19	562,81	1,36	0,11	3,5%	4,2%	90,3%	80,6%	9,7%	55,0%
	E		0,18	1,41				3,1%	3,4%	91,7%	58,8%	32,9%	
k4a (Par)	P	0,0979	0,42	2,45	510,93	1,24	0,10	7,2%	25,5%	42,7%	29,7%	13,0%	57,1%
	E		0,41	1,72				6,2%	20,4%	48,2%	48,2%	0,0%	
k5 (Par)	P	0,0123	0,38	0,31	4078,30	9,86	0,82	40,2%	17,2%	26,0%	5,5%	20,4%	54,5%
	E		0,24	0,21				27,7%	12,6%	48,4%	29,6%	18,8%	
k5b (Bit)	P	0,0065	0,25	0,16	7672,24	18,55	1,55	69,6%	13,9%	9,4%	1,1%	8,3%	67,8%
	E		0,28	0,12				49,8%	7,6%	33,9%	3,3%	30,6%	
k5c (Odd)	P	0,0078	0,33	0,20	6403,69	15,48	1,29	56,0%	10,2%	22,3%	3,3%	19,0%	60,2%
	E		0,28	0,14				43,7%	10,9%	31,5%	8,4%	23,1%	

d) Analiza stopnia efektywności wykorzystania zasobów i zidentyfikowane wąskie gardła

- **Algorytm dzielenia:** Wariant k1 jest najwolniejszy we wszystkich zakresach, ponieważ wykonuje dużą liczbę operacji dzielenia modulo w jednej sekwencyjnej pętli. Dla pełnego zakresu osiąga jedynie 1,5 M/s. Zrównoleglenie tego samego podejścia w k2 zwiększa prędkość do 17,05 M/s, jednak nadal jest to znacznie mniej niż w wariantach sitowych. Ograniczeniem pozostaje koszt operacji arytmetycznych i zmienny czas sprawdzania poszczególnych liczb.
- **Wpływ porcji zadań w k2_chunk:** Wariant k2_chunk nie zmienia algorytmu, ale ogranicza narzut dynamicznego przydziału zadań przez grupowanie iteracji w porcje. Dla pełnego zakresu poprawa względem k2 jest niewielka: 5,846 s wobec 5,866 s. Dla zakresu górnego różnica jest większa: 3,481 s wobec 4,080 s. Wskazuje to, że dobór porcji może poprawiać równowagę pracy i zmniejszać narzut planisty, ale nie usuwa głównego ograniczenia wariantu dzielenia.
- **Znaczenie lokalności w wariantach sekwencyjnych k3 i k3a:** Wariant k3 jest znacznie szybszy od dzielenia, ponieważ zamiast wielokrotnego sprawdzania dzielników wykreśla wielokrotności. Jego głównym ograniczeniem jest jednak słaba lokalność dostępu do `result`. Dla pełnego zakresu k3 ma *Memory Bound* na rdzeniach P równy 95,9% i czas 0,6308 s. Wariant blokowy k3a skraca ten czas około 6-krotnie do 0,1048 s, ponieważ ogranicza aktualnie modyfikowany fragment tablicy. Mimo tego k3a nadal jest wariantem sekwencyjnym i wykorzystuje fizyczne rdzenie na poziomie około 8,1%.
- **Sito funkcyjne k4 i k4a:** Warianty k4 i k4a równoległą pętlę po dzielnikach, a więc stosują podział funkcyjny. Dla pełnego zakresu k4 osiąga średni czas 0,2194 s, a k4a 0,2204 s. Oba warianty są wolniejsze od sekwencyjnego wariantu blokowego k3a, a dodatkowo pozostają problematyczne ze względu na współbieżne dostępy do tablicy `result`.

W wariantach k4 dominuje ograniczenie pamięciowe: *Memory Bound* na rdzeniach P wynosi 88,5%, a *Store Bound* 84,2%. Oznacza to, że głównym problemem są bezwarunkowe, powtarzane zapisy do tablicy wynikowej oraz koszt obsługi spójności cache. Jednocześnie kod k4 ma prosty przebieg wewnętrznej pętli, ponieważ wykonuje bezwarunkową instrukcję `result[j-m] = false`. W raporcie VTune widać, że koszt nieudanej spekulacji jest w tym wariantie znikomy: *Bad Speculation* na rdzeniach P wynosi 0,0%, a *Branch Mispredict* 0,1%.

W wariantach k4a dodatkowy warunek `if (result[j-m])` ogranicza liczbę powtórnych zapisów i zmniejsza *Memory Bound* na rdzeniach P do 43,5%. Zysk pamięciowy nie przełożył się jednak na krótszy czas wykonania, ponieważ wariant wykonuje znacznie więcej instrukcji: około 1,812 G na jedno wykonanie wobec około 0,962 G w k4. Dodatkowo w k4a pojawia się rozgałęzienie zależne od aktualnego stanu tablicy `result`. Procesor próbuje przewidywać wynik tego warunku i wykonywać kod spekulacyjnie, ale raport VTune pokazuje dużą liczbę błędnych przewidywań: *Bad Speculation* na P-core wynosi 20,6%, a *Branch Mispredict* 19,7%. Na rdzeniach E-core wartości te są jeszcze nieco wyższe i wynoszą odpowiednio

23,9% oraz 23,1%. Oznacza to, że część pracy procesora była wykonywana na błędnie przewidzianej ścieżce i musiała zostać odrzucona. Dlatego k4a, mimo niższego ograniczenia pamięciowego, nie wykonał się szybciej od k4.

- **Sito domenowe k5:** Wariant k5 eliminuje główny problem k4, ponieważ wątki pracują na rozłącznych blokach tablicy wynikowej. Dla pełnego zakresu czas spada do 0,0234 s, a prędkość rośnie do 4267,77 M/s. W porównaniu z k3a daje to przyspieszenie 4,47. Wariant ten ma znacznie niższy udział *Memory Bound* na rdzeniach P niż k4, ponieważ zapis jest lokalny dla bloku i nie występuje wielowątkowe wykreślanie tych samych elementów.
- **Wariant bitowy k5b:** Najszybszym wariantem we wszystkich zakresach jest k5b. Dla pełnego zakresu uzyskuje 0,0122 s i 8218,28 M/s, czyli około 1,93 razy większą prędkość niż k5. Wynika to z około ośmiokrotnego zmniejszenia rozmiaru tablicy wynikowej. *Memory Bound* na rdzeniach P spada do 1,6%, a *Retiring* rośnie do 70,5%. Oznacza to, że procesor wykonuje znacznie większą część użytecznej pracy, a nie oczekuje na dane z pamięci.
- **Wariant liczb nieparzystych k5c:** Wariant k5c zmniejsza liczbę przechowywanych elementów prawie dwukrotnie przez pominięcie liczb parzystych. Dla pełnego zakresu osiąga 0,0156 s i 6406,56 M/s, czyli jest szybszy od k5, ale wolniejszy od k5b. Powodem jest dodatkowy koszt przeliczania indeksów i wyznaczania kolejnych nieparzystych wielokrotności. Widać to w większym udziale *Core Bound* względem k5b: 23,7% wobec 8,5% na rdzeniach P dla pełnego zakresu.
- **Efektywne wykorzystanie rdzeni:** Największe wykorzystanie rdzeni fizycznych nie zawsze oznacza najkrótszy czas. Dla pełnego zakresu k2_chunk osiąga wysokie wykorzystanie 76,3%, ale jego czas wynosi 5,846 s. Z kolei k5b ma wykorzystanie 69,7%, ale kończy obliczenia w 0,0122 s. Oznacza to, że samo wykorzystanie rdzeni nie wystarcza do oceny jakości wariantu; kluczowe są także lokalność danych, liczba operacji pamięciowych i rodzaj podziału pracy.
- **Ocena wariantów 3f:** Wariant k2_chunk bada wpływ rozmiaru porcji na szeregowanie dynamiczne. k5b pokazuje wpływ zmiany reprezentacji danych na pamięć podręczną i liczbę transferów danych między poziomami pamięci. k5c pokazuje wpływ zmniejszenia dziedziny obliczeń przez pominięcie liczb parzystych. Największą poprawę efektywności daje k5b, ponieważ ogranicza zarówno rozmiar danych, jak i liczbę linii cache potrzebnych do przechowania tablicy wynikowej.

e) Szczegółowa analiza ograniczeń podsystemu pamięci

Oprócz podstawowych metryk przedstawionych w tabelach zbiorczych przeanalizowano również szczegółowe podmetryki dostępne w Intel VTune. Pozwalają one wskazać, który element hierarchii pamięci powodował opóźnienia: pamięć podręczna L1, L2, L3, pamięć operacyjna DRAM, bufor translacji adresów TLB lub mechanizm obsługi zapisów. Analizę wykonano dla pełnego zakresu $\langle 2, 10^8 \rangle$, ponieważ w tej instancji rozmiar danych i liczba operacji zapisu są największe.

Należy zaznaczyć, że metryki szczegółowe takie jak *L1 Bound*, *L2 Bound*, *L3 Bound*, *DRAM Bound*, *Store Bound*, *DTLB Store Overhead* oraz *FB Full* są metrykami diagnostycznymi raportowanymi przez VTune. Nie należy ich sumować bezpośrednio do wartości *Memory Bound*, ponieważ część z nich jest liczona względem cykli zegara, a część względem kategorii modelu TMA. Ich zadaniem jest wskazanie dominującego źródła opóźnień.

Tabela 7: Szczegółowe metryki podsystemu pamięci dla wybranych wariantów, P-core, zakres $\langle 2, 10^8 \rangle$

Wariant	Mem.B	L1 Bound	L2 Bound	L3 Bound	DRAM Bound	Store Bound	DTLB Store	FB Full
k3	95,9%	0,0%	1,1%	1,1%	0,4%	95,4%	23,7%	54,5%
k4	88,5%	0,0%	0,8%	2,4%	1,6%	86,1%	68,1%	80,5%
k4a	43,5%	6,8%	0,0%	5,5%	54,7%	0,0%	15,2%	50,6%
k5	8,4%	4,7%	0,0%	2,4%	0,1%	13,4%	12,2%	3,4%
k5b	1,6%	6,0%	0,0%	0,8%	0,1%	0,5%	0,6%	0,2%
k5c	3,2%	5,2%	0,1%	1,6%	0,1%	2,0%	10,5%	1,6%

- **Wariant k3:** Wariant sekwencyjny bez blokowania jest silnie ograniczony przez operacje zapisu do tablicy *result*. Wartość *Memory Bound* wynosi 95,9%, a *Store Bound* 95,4%. Oznacza to, że dominującym problemem nie jest samo wykonywanie obliczeń, lecz obsługa zapisów do dużej tablicy wynikowej. Dodatkowo *FB Full* wynosi 54,5%, co wskazuje na częste zapełnianie buforów związanych z operacjami pamięciowymi.
- **Wariant k4:** Wariant funkcyjny jest najbardziej problematyczny z punktu widzenia zapisów współbieżnych. *Memory Bound* wynosi 88,5%, a *Store Bound* 86,1%. Szczególnie istotne są wartości *DTLB Store Overhead* równe 68,1% oraz *FB Full* równe 80,5%. Oznacza to, że procesor często napotykał opóźnienia związane z translacją adresów przy zapisach oraz zapełnianiem buforów pamięciowych. Jest to zgodne ze

strukturą kodu, w której wiele wątków zapisuje do tej samej dużej tablicy **result**, często do tych samych lub sąsiednich linii pamięci podręcznej.

- **Wariant k4a:** Dodanie warunku `if (result[j-m])` wyraźnie ogranicza koszt powtórnych zapisów. *Store Bound* spada z 86,1% w k4 do 0,0% w k4a. Jednocześnie *Memory Bound* nadal pozostaje zauważalne i wynosi 43,5%. Wynika to z faktu, że zapis został częściowo zastąpiony dodatkowymi odczytami i instrukcją warunkową. Wysokie wartości *DRAM Bound* oraz *FB Full* wskazują, że mimo ograniczenia liczby zapisów kod nadal jest wrażliwy na dostęp do danych i na konflikty w podsystemie pamięci.
- **Wariant k5:** Przejście na podział domenowy znacząco zmniejsza ograniczenia pamięciowe. *Memory Bound* spada do 8,4%, ponieważ każdy wątek pracuje na własnym, rozłącznym bloku tablicy wynikowej. Ogranicza to konflikty między rdzeniami i zmniejsza liczbę unieważnień linii cache. Nadal widoczny jest pewien koszt zapisów, ponieważ *Store Bound* wynosi 13,4%, ale jest on zdecydowanie mniejszy niż w wariantach k3 i k4.
- **Wariant k5b:** Reprezentacja bitowa najbardziej ogranicza obciążenie pamięci. *Memory Bound* spada do 1,6%, *Store Bound* do 0,5%, a *DTLB Store Overhead* do 0,6%. Wynika to z około ośmiokrotnego zmniejszenia rozmiaru tablicy wynikowej. Mniejszy zbiór danych oznacza mniej używanych linii cache i mniej stron pamięci, a więc również mniejsze koszty translacji adresów.
- **Wariant k5c:** Wariant przechowujący tylko liczby nieparzyste również ogranicza obciążenie pamięci, choć słabiej niż k5b. *Memory Bound* wynosi 3,2%, a *Store Bound* 2,0%. Redukcja danych jest tutaj około dwukrotna, a nie ośmiokrotna jak w wariancie bitowym. Z tego powodu k5c poprawia wynik względem k5, ale nie osiąga tak niskiego udziału ograniczeń pamięciowych jak k5b. Widoczny pozostaje koszt *DTLB Store Overhead* równy 10,5%, związany z zapisem do tablicy wynikowej.

Punkt 4: Wnioski

- **Porównanie ogólnej jakości badanych podejść:** Największy wpływ na czas wykonania miała nie sama obecność zrównoleglenia, lecz wybór algorytmu i sposób dostępu do danych. Wariant dzielenia sekwencyjnego k1 dla pełnego zakresu $\langle 2, 10^8 \rangle$ osiągnął jedynie 1,50 M/s, natomiast równoległy wariant dzielenia k2 osiągnął 17,05 M/s. Oznacza to wzrost prędkości około 11,4 razy, ale nadal wynik ten jest znacznie słabszy niż w wariantach opartych na wykreślaniu wielokrotności. Wariant sekwencyjnego sita blokowego k3a osiągnął 954,12 M/s, czyli około 56 razy większą prędkość niż k2. Pokazuje to, że zmiana sposobu przetwarzania z dzielenia na sito oraz poprawa lokalności danych były ważniejsze niż samo zrównoleglenie kosztownej metody dzielenia.
- **Wpływ lokalności dostępu do danych:** Porównanie wariantów k3 i k3a pokazuje, że lokalność dostępu do danych jest jednym z kluczowych czynników wydajności. Wariant k3 dla pełnego zakresu uzyskał 158,52 M/s, natomiast k3a 954,12 M/s. Oznacza to wzrost prędkości około 6-krotny po wprowadzeniu przetwarzania blokowego. Wynika to z faktu, że k3a ogranicza aktualnie modyfikowany fragment tablicy **result**, dzięki czemu zmniejsza liczbę kosztownych odwołań do dalszych poziomów hierarchii pamięci. Potwierdzają to wyniki VTune: k3 ma bardzo wysoki udział *Memory Bound*, natomiast k3a mimo że nadal jest wariantem sekwencyjnym, lepiej wykorzystuje pamięć podręczną.
- **Ocena podejścia funkcyjnego w wariantach k4 i k4a:** Warianty k4 i k4a równoległą pętlę po dzielnikach, czyli stosując podział funkcyjny. Podejście to okazało się niekorzystne dla analizowanego problemu. Dla pełnego zakresu k4 osiągnął 455,83 M/s, a k4a 453,68 M/s, czyli oba warianty były około 2,1 razy wolniejsze od sekwencyjnego wariantu blokowego k3a. Przyczyną jest intensywny, współbieżny zapis do tej samej tablicy **result**. Warianty te generują wyścigi na danych, wysokie ryzyko false sharingu oraz duży koszt utrzymania spójności cache. W k4 *Memory Bound* na rdzeniach P wyniósł 88,5%, co pokazuje, że procesor przez znaczną część czasu był ograniczany przez podsystem pamięci, a nie przez same obliczenia.
- **Porównanie k4 i k4a:** Dodanie warunku `if (result[j-m])` w wariancie k4a ograniczyło liczbę powtórnych zapisów do tablicy wynikowej. Widać to po spadku *Memory Bound* na rdzeniach P z 88,5% w k4 do 43,5% w k4a. Z punktu widzenia samego systemu pamięci wariant k4a wygląda więc korzystniej, ponieważ wykonuje mniej bezwarunkowych zapisów do współdzielonej tablicy.
Nie oznacza to jednak automatycznie krótszego czasu wykonania. Wariant k4a wykonuje około 1,812 G instrukcji na jedno wykonanie programu, podczas gdy k4 wykonuje około 0,962 G. Dodatkowo raport VTune pokazuje, że k4a jest silnie obciążony przez nieudaną spekulację procesora. Dla P-core metryka *Bad Speculation* rośnie z 0,0% w k4 do 20,6% w k4a, a *Branch Mispredict* z 0,1% do 19,7%. Dla E-core wzrost jest podobny: *Bad Speculation* rośnie z 2,5% do 23,9%, a *Branch Mispredict* z 0,9% do 23,1%.
Przyczyną jest dodatkowy warunek zależny od danych w tablicy **result**. W k4 wewnętrzna pętla wykonuje bezwarunkowy zapis `result[j-m] = false`, natomiast w k4a procesor musi przewidywać, czy warunek

if (result[j-m]) będzie spełniony. Ponieważ tablica **result** jest modyfikowana równolegle przez wiele wątków, stan kolejnych elementów jest trudniejszy do przewidzenia. Błędna predykcja powoduje odrzucenie pracy wykonanej spekulacyjnie i ponowne skierowanie wykonania na właściwą ścieżkę. W efekcie **k4a** poprawia charakterystyki pamięciowe, ale traci czas na dodatkowe odczyty, większą liczbę instrukcji oraz błędne przewidywania rozgałęzień. Dlatego jego czas 0,2204 s jest minimalnie gorszy niż czas **k4**, równy 0,2194 s.

- **Najlepsze podejście równoległe:** Najbardziej efektywnym podstawowym wariantem równoległym jest **k5**, czyli równoległe sito domenowe z lokalnością danych. Dla pełnego zakresu osiągnęło ono 4267,77 M/s i czas 0,0234 s. W porównaniu z **k3a** daje to przyspieszenie $S_p = 4,47$ oraz efektywność $E_p = 0,37$. Wariant **k5** jest około 9,4 razy szybszy od **k4** i około 4,5 razy szybszy od sekwencyjnego **k3a**. Przyczyną jest domenowy podział pracy: każdy wątek otrzymuje osobny blok tablicy **result**, więc zapisy są lokalne, nie występują wyścigi na tych samych elementach, a false sharing ogranicza się głównie do granic bloków.
- **Najlepszy wariant spośród wszystkich badanych kodów:** Najlepszy wynik we wszystkich badanych zakresach uzyskał wariant **k5b**, czyli równoległe sito domenowe z reprezentacją bitową. Dla pełnego zakresu osiągnął 8218,28 M/s i czas 0,0122 s. Jest to około 1,93 razy szybciej niż **k5**, około 8,6 razy szybciej niż **k3a** oraz ponad 5400 razy szybciej niż **k1**. Wariant ten łączy poprawny domenowy podział pracy z około ośmiokrotnym zmniejszeniem rozmiaru tablicy wynikowej. Dzięki temu zmniejsza liczbę używanych linii cache, ogranicza liczbę transferów danych między poziomami pamięci i redukuje narzut translacji adresów. Potwierdza to niski udział *Memory Bound* równy 1,6% na rdzeniach P oraz wysoki udział *Retiring* równy 70,5%.
- **Ocena wariantu k5c:** Wariant **k5c** również poprawia wydajność względem **k5**, ponieważ pomija liczby parzyste i zmniejsza liczbę elementów tablicy wynikowej prawie dwukrotnie. Dla pełnego zakresu osiąga 6406,56 M/s, czyli około 1,5 razy więcej niż **k5**. Jest jednak wolniejszy od **k5b**, ponieważ redukcja danych jest mniejsza, a kod wymaga dodatkowego przeliczania indeksów oraz wyznaczania tylko nieparzystych wielokrotności. Widać to po większym udziale *Core Bound*: 23,7% w **k5c** wobec 8,5% w **k5b**. Oznacza to, że w **k5c** część zysku z mniejszej liczby danych jest tracona na bardziej złożoną logikę obliczeń.
- **Najgorsze podejście:** Najgorszym wariantem pod względem prędkości był **k1**, czyli sekwencyjne dzielenie. Jego głównym ograniczeniem jest bardzo duża liczba operacji modulo i brak wykorzystania równoległości. Najgorszym wariantem równoległym pod względem relacji kosztu do korzyści pozostaje **k2**, ponieważ mimo dobrego wykorzystania rdzeni fizycznych osiąga znacznie niższą prędkość niż warianty sitowe. Warianty **k4** i **k4a** są z kolei problematyczne nie tylko wydajnościowo, ale także poprawnościowo, ponieważ zawierają wyścigi na tablicy wynikowej.
- **Efektywność zrównoleglenia:** Efektywność równoległa zależała od tego, czy podział pracy był zgodny z charakterem danych. Wariant **k2** ma dużo zadań i dobrze zajmuje rdzenie, ale każde zadanie jest kosztowne obliczeniowo i zmienne czasowo, dlatego przyspieszenie względem najlepszego wariantu sekwencyjnego jest bardzo małe. Warianty **k4** i **k4a** używają wielu wątków, ale przez współdzielone zapisy do **result** tracą wydajność na pamięci i spójności cache. Najlepszą jakość zrównoleglenia daje **k5**, ponieważ praca jest dzielona na rozłączne bloki danych. Warianty **k5b** i **k5c** osiągają jeszcze lepsze wyniki, ale ich przyspieszenie nie wynika wyłącznie z równoległości, lecz także ze zmiany reprezentacji danych i zmniejszenia liczby przetwarzanych elementów.
- **Wąskie gardła poszczególnych metod:** Warianty dzielenia **k1**, **k2** i **k2_chunk** są ograniczane głównie przez koszt operacji arytmetycznych oraz zmienny czas sprawdzania kolejnych liczb. Wariant **k3** jest ograniczany przez słabą lokalność dostępu do tablicy wynikowej. Szczegółowe metryki VTune pokazują, że w wariantach **k3** i **k4** głównym problemem nie jest sam dostęp do pamięci DRAM, lecz przede wszystkim obsługa zapisów, translacja adresów dla zapisu oraz zapełnianie buforów pamięciowych. Warianty **k4** i **k4a** dodatkowo tracą wydajność przez zapisy do współdzielonej tablicy, wyścigi, false sharing i mechanizmy spójności cache. Wariant **k5** ogranicza te problemy przez domenowy podział tablicy wynikowej, a **k5b** dodatkowo zmniejsza rozmiar danych dzięki reprezentacji bitowej. Wariant **k5c** również zmniejsza liczbę danych, kosztem bardziej złożonego przeliczania indeksów.
- **Wpływ architektury hybrydowej P-core/E-core:** Procesor Intel Core i5-12500H ma rdzenie o różnej wydajności: rdzenie P-core są szybsze i lepiej nadają się do ciężkich zadań jednowątkowych, natomiast rdzenie E-core zwiększają łączną przepustowość przy dużej liczbie niezależnych zadań. Potwierdza to porównanie uruchomień na całym procesorze i wyłącznie na rdzeniach P-core. Wariant **k2** wykonuje się odpowiednio w czasie 5,866 s oraz 12,233 s, a **k2_chunk** w czasie 5,846 s oraz 12,239 s. Oznacza to, że mimo niższej wydajności pojedynczego rdzenia E-core, ich obecność wyraźnie zwiększa całkowitą przepustowość dla wariantów z dużą liczbą niezależnych iteracji.
Wpływ architektury hybrydowej jest mniejszy dla wariantów ograniczonych przez pamięć. Warianty **k4** i **k4a** przyspieszają po użyciu całego procesora, ale nie proporcjonalnie do liczby rdzeni, ponieważ ich głów-

nym ograniczeniem są współbieżne zapisy do tablicy `result`, false sharing i koszt utrzymania spójności cache. Warianty domenowe `k5`, `k5b` i `k5c` korzystają z dodatkowych rdzeni lepiej, ponieważ dzielą pracę na niezależne bloki danych.

- **Zalety wykonywania kodów na procesorze hybrydowym:** Główną zaletą architektury hybrydowej jest możliwość zwiększenia całkowitej przepustowości programu przez wykorzystanie dodatkowych rdzeni E-core. Jest to szczególnie korzystne dla wariantów mających dużo niezależnych zadań, takich jak `k2`, `k2_chunk`, `k5`, `k5b` i `k5c`. Dynamiczny przydział pracy pozwala szybszym rdzeniom P-core wykonywać więcej zadań, a wolniejszym rdzeniom E-core nadal uczestniczyć w obliczeniach bez ręcznego przypisywania pracy do typów rdzeni.

Dla wariantów blokowych dodatkowe rdzenie E-core zwiększają liczbę jednocześnie przetwarzanych bloków. Najlepiej widać to w wariancie `k5b`, który przy użyciu całego procesora osiąga 0,0122 s, natomiast na samych P-core 0,0244 s. Oznacza to, że połączenie małej reprezentacji danych z dużą liczbą rdzeni bardzo dobrze zwiększa przepustowość przetwarzania.

- **Wady wykonywania kodów na procesorze hybrydowym:** Wadą architektury hybrydowej jest trudniejsze przewidywanie równowagi obciążenia. Rdzenie P-core i E-core mają różną wydajność, dlatego czas zakończenia pętli równoległej może zależeć od tego, jakie zadania trafiają na wolniejsze rdzenie E-core. Problem ten jest szczególnie istotny dla wariantów o nierównym koszcie pojedynczych zadań, takich jak `k2`, `k4` i `k4a`.

Dodatkowe rdzenie nie gwarantują proporcjonalnego przyspieszenia, jeżeli wąskim gardłem jest pamięć. Warianty `k4` i `k4a` są szybsze na całym procesorze niż na samych P-core, ale nadal pozostają znacznie wolniejsze od wariantów domenowych. Przyczyną nie jest brak mocy obliczeniowej, lecz sposób dostępu do wspólnej tablicy `result`, który powoduje konflikty pamięciowe, powtarzne zapisy i koszt spójności cache. Dodatkowym utrudnieniem jest interpretacja metryk VTune, ponieważ P-core i E-core mają inną charakterystykę wykonania.

- **Znaczenie dynamicznego szeregowania na procesorze P/E:** Dynamiczne szeregowanie jest szczególnie uzasadnione na procesorze hybrydowym, ponieważ pozwala dopasować ilość wykonanej pracy do rzeczywistej wydajności rdzeni. Szybsze rdzenie P-core mogą pobierać więcej zadań, a wolniejsze E-core mniej, co ogranicza ryzyko nierównomiernego obciążenia. Jest to korzystne zwłaszcza w wariantach, w których koszt iteracji jest zmienny.

Porównanie uruchomień pokazuje jednak, że samo `schedule(dynamic)` nie rozwiązuje wszystkich problemów. W wariantach dzielenia dodatkowe rdzenie E-core wyraźnie skracają czas wykonania, ponieważ liczba iteracji jest bardzo duża. W wariantach `k4` i `k4a` efekt jest mniejszy, ponieważ nawet dynamiczny przydział pracy nie usuwa konfliktów wynikających ze współbieżnych zapisów do tej samej tablicy. Najlepiej dynamiczne szeregowanie współpracuje z podziałem domenowym, gdzie zadania odpowiadają rozłącznym blokom danych i nie powodują wyścigów na tych samych elementach wyniku.

- **Możliwości dalszej poprawy:** Dalszą poprawę można uzyskać przez połączenie zalet wariantów `k5b` i `k5c`, czyli zastosowanie reprezentacji bitowej tylko dla liczb nieparzystych. Warto również badać inne wartości `blockSize`, ponieważ zbyt mały blok zwiększa narzut planisty OpenMP, a zbyt duży blok pogarsza zrównoważenie pracy, szczególnie na rdzeniach P/E. Możliwe jest także sprawdzenie przypięcia wątków do rdzeni przez ustawienia `OMP_PROC_BIND` i `OMP_PLACES`, aby ograniczyć migrację wątków między rdzeniami oraz powtarzalniej porównać zachowanie P-core i E-core.

- **Wniosek końcowy:** Najlepszą strategią dla badanego problemu jest równoległe sito domenowe z wysoką lokalnością dostępu do danych i małą reprezentacją tablicy wynikowej. Najlepszy wynik uzyskuje więc wariant, który nie tylko wykorzystuje równoległość, ale również minimalizuje ruch pamięciowy, ogranicza konflikty zapisu i poprawia lokalność danych. W badanych wariantach rolę tę najlepiej spełnia `k5b`, ponieważ łączy domenowy podział pracy z bitową reprezentacją tablicy wynikowej.